

vLLM专题（十六）-推测性解码（Speculative Decoding）

原创

AI专题精讲

已于 2025-03-19 11:01:12 修改

阅读量139

收藏

点赞数

版权

分类专栏：

大模型专题系列

 文章标签：

人工智能

 大模型专题系列 专栏收录该内容

 您已是超级会员，正在免费阅读会员专享内容 [查看更多超级会员权益](#)

警告

请注意，vLLM 中的推测性解码尚未优化，并且通常不会对所有提示数据集或采样参数带来令牌间延迟的减少。优化工作正在进行中，相关进展可以通过此链接跟踪：[问题 #4630](#)。

警告

目前，vLLM 中的推测性解码与管道并行性不兼容。

本文档展示了如何在 vLLM 中使用推测性解码。推测性解码是一种提高内存受限的 LLM 推理中令牌间延迟的技术。

一、使用草拟模型进行推测

以下代码配置 vLLM 以离线模式使用推测性解码，并使用草拟模型每次推测 5 个token。

¥49.90

¥99.00

92 篇文章

已订阅

8折续费

```

from vllm import LLM, SamplingParams

prompts = [
    "The future of AI is",
]
sampling_params = SamplingParams(temperature=0.8, top_p=0.95)

llm = LLM(
    model="facebook/opt-6.7b",
    tensor_parallel_size=1,
    speculative_model="facebook/opt-125m",
    num_speculative_tokens=5,
)
outputs = llm.generate(prompts, sampling_params)

for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(f"Prompt: {
        prompt!r}, Generated text: {
        generated_text!r}")

```

要在在线模式下执行相同操作，请启动服务器：

```

python -m vllm.entrypoints.openai.api_server --host 0.0.0.0 --port 8000 --model facebook/
opt-6.7b \
--seed 42 -tp 1 --speculative_model facebook/opt-125m \
--num_speculative_tokens 5 --gpu_memory_utilization 0.8

```

然后使用客户端：

```

from openai import OpenAI

# Modify OpenAI's API key and API base to use vLLM's API server.
openai_api_key = "EMPTY"
openai_api_base = "http://localhost:8000/v1"

client = OpenAI(
    # defaults to os.environ.get("OPENAI_API_KEY")
    api_key=openai_api_key,
    base_url=openai_api_base,
)

models = client.models.list()
model = models.data[0].id

# Completion API
stream = False
completion = client.completions.create(
    model=model,
    prompt="The future of AI is",
    echo=False,
    n=1,
    stream=stream,
)

print("Completion results:")
if stream:
    for c in completion:
        print(c)
else:
    print(completion)

```

二、通过匹配提示中的 n-gram 进行推测解码

以下代码配置 vLLM 使用推测解码，其中提案通过匹配提示中的 n-gram 生成。更多信息请阅读[此讨论](#)。

```
from vllm import LLM, SamplingParams

prompts = [
    "The future of AI is",
]
sampling_params = SamplingParams(temperature=0.8, top_p=0.95)

llm = LLM(
    model="facebook/opt-6.7b",
    tensor_parallel_size=1,
    speculative_model="[ngram]",
    num_speculative_tokens=5,
    ngram_prompt_lookup_max=4,
)
outputs = llm.generate(prompts, sampling_params)

for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(f"Prompt: {
        prompt!r}, Generated text: {
        generated_text!r}")
```

三、使用 MLP 推测器进行推测解码

以下代码配置 vLLM 使用推测解码，其中提案由基于上下文向量和采样 token 的草稿模型生成。更多信息请参阅[此博客](#)或[此技术报告](#)。

```

from vllm import LLM, SamplingParams

prompts = [
    "The future of AI is",
]
sampling_params = SamplingParams(temperature=0.8, top_p=0.95)

llm = LLM(
    model="meta-llama/Meta-Llama-3.1-70B-Instruct",
    tensor_parallel_size=4,
    speculative_model="ibm-ai-platform/llama3-70b-accelerator",
    speculative_draft_tensor_parallel_size=1,
)
outputs = llm.generate(prompts, sampling_params)

for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(f"Prompt: {
        prompt!r}, Generated text: {
        generated_text!r}")

```

注意

目前，这些推测模型需要在不使用张量并行的情况下运行，尽管主模型可以使用张量并行运行（见上例）。由于推测模型相对较小，我们仍然可以看到显著的加速效果。然而，此限制将在未来的版本中修复。

可用的推测模型

以下类型的推测模型可在 HF hub 上找到：

- llama-13b-accelerator
- llama3-8b-accelerator
- codellama-34b-accelerator
- llama2-70b-accelerator
- llama3-70b-accelerator
- granite-3b-code-instruct-accelerator
- granite-8b-code-instruct-accelerator
- granite-7b-instruct-accelerator
- granite-20b-code-instruct-accelerator

四、使用基于 **EAGLE** 的草拟模型进行推测

以下代码配置 vLLM 以使用推测性解码，其中提案由基于 EAGLE（用于提高语言模型效率的外推算法）的草拟模型生成。

```
from vllm import LLM, SamplingParams

prompts = [
    "The future of AI is",
]
sampling_params = SamplingParams(temperature=0.8, top_p=0.95)

llm = LLM(
    model="meta-llama/Meta-Llama-3-8B-Instruct",
    tensor_parallel_size=4,
    speculative_model="yuhuili/EAGLE-LLaMA3-Instruct-8B",
    speculative_draft_tensor_parallel_size=1,
)

outputs = llm.generate(prompts, sampling_params)

for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(f"Prompt: {
        prompt!r}, Generated text: {
        generated_text!r}")
```

使用基于 EAGLE 的草拟模型时需要考虑的几点重要事项：

EAGLE 草拟模型：HF 仓库中提供的 EAGLE 草拟模型应该能够在 PR 12304 后被 vLLM 直接加载和使用。如果你使用的是 vLLM 版本低于 PR 12304，请使用脚本将推测性模型转换，并指定 `speculative_model="path/to/modified/eagle/model"`。如果在使用最新版本的 vLLM 时仍然遇到权重加载问题，请留言或提交问题。

关于 EAGLE 草拟模型的张量并行性：基于 EAGLE 的草拟模型需要在没有张量并行性的情况下运行（即 `speculative_draft_tensor_parallel_size` 设置为 1），尽管主模型可以使用张量并行性运行（请参见上面的示例）。

速度提升问题：使用基于 EAGLE 的推测器时，观察到的加速效果低于此处参考实现中报告的加速效果。此问题正在调查中，并已在此跟踪：[vllm-project/vllm#9565](https://github.com/vllm-project/vllm/issues/9565)。

EAGLE 草拟模型的可用性：多种 EAGLE 草拟模型可在 Hugging Face hub 上找到。

Base Model	EAGLE on Hugging Face	# EAGLE Parameters
Vicuna-7B-v1.3	yuhuili/EAGLE-Vicuna-7B-v1.3	0.24B
Vicuna-13B-v1.3	yuhuili/EAGLE-Vicuna-13B-v1.3	0.37B
Vicuna-33B-v1.3	yuhuili/EAGLE-Vicuna-33B-v1.3	0.56B
LLaMA2-Chat 7B	yuhuili/EAGLE-llama2-chat-7B	0.24B
LLaMA2-Chat 13B	yuhuili/EAGLE-llama2-chat-13B	0.37B
LLaMA2-Chat 70B	yuhuili/EAGLE-llama2-chat-70B	0.99B
Mixtral-8x7B-Instruct-v0.1	yuhuili/EAGLE-mixtral-instruct-8x7B	0.28B
LLaMA3-Instruct 8B	yuhuili/EAGLE-LLaMA3-Instruct-8B	0.25B
LLaMA3-Instruct 70B	yuhuili/EAGLE-LLaMA3-Instruct-70B	0.99B
Qwen2-7B-Instruct	yuhuili/EAGLE-Qwen2-7B-Instruct	0.26B
Qwen2-72B-Instruct	yuhuili/EAGLE-Qwen2-72B-Instruct	1.05B

五、推测性解码的无损保证

在 vLLM 中，推测性解码旨在提高推理效率，同时保持准确性。本节讨论推测性解码的无损保证，将其分解为三个关键领域：

- 理论无损性** - 推测性解码采样在硬件数值精度的限制下，理论上是无损的。浮点错误可能导致输出分布的轻微变化，正如在《使用推测性采样加速大规模语言模型解码》中所讨论的那样。
- 算法无损性** - vLLM 对推测性解码的实现进行了算法验证，确保其无损。关键验证测试包括：

拒绝采样器收敛性：确保来自 vLLM 拒绝采样器的样本与目标分布一致。查看测试代码

贪婪采样相等性：确认使用推测性解码的贪婪采样与不使用推测性解码的贪婪采样相匹配。这验证了 vLLM 的推测性解码框架，当与 vLLM 前向传递和 vLLM 拒绝采样器结合时，提供了无损保证。几乎所有在 `tests/spec_decode/e2e` 中的测试都使用此断言实现来验证这一属性。

- 3. vLLM 对数概率稳定性** - vLLM 目前不保证令牌对数概率（logprobs）的稳定性。这可能导致相同请求在不同运行之间产生不同的输出。有关更多细节，请参阅 FAQ 中名为“Can the output of a prompt vary across runs in vLLM?”的部分。

尽管 vLLM 力求确保推测性解码的无损性，但由于以下因素，使用和不使用推测性解码时生成的输出可能会有所不同：

浮点精度：硬件数值精度的差异可能导致输出分布的轻微不一致。

批量大小和数值稳定性：批量大小的变化可能导致对数概率（logprobs）和输出概率的变化，这可能是由于批处理操作中的非确定性行为或数值不稳定性引起的。

有关缓解策略，请参阅 FAQ 中名为“Can the output of a prompt vary across runs in vLLM?”的条目。



AI专题精讲

已关注



0



0



0



分享



专栏目录

已订阅